Text-to-Speech with Transformer and Codec Language Models

Applied Deep Learning — Chapter 24

Yin Yang

Foundation Books Project

Roadmap

Running Example and Goals

From ASR to TTS

- ASR: audio $\rightarrow$ text. TTS: text and voice info $\rightarrow$ waveform.
- Reversal sounds simple but changes the problem:
  - a transcript can be exactly correct;
  - a waveform has *many* acceptable versions (rate, pitch, style).
- "Sounds like me" is a hypothesis, not a result.

Reuse audio vocabulary from ASR; add pronunciation, prosody, vocoders, codec tokens, and acoustic prompts.

The Running Example: Voice Cloning

A reader records 6 s of clean reference audio, writes its exact transcript, and asks a TTS system to speak three new sentences in the same voice.

Inputs target text y, language and style c, reference clip x^{ref}, reference transcript.

Output waveform $\hat{x}$ written to a WAV file.

Question is the waveform useful evidence of *controlled* speech generation?

Modern case study: Qwen3-TTS — multilingual, controllable, streaming.

Project Artifacts Before Generation

Artifact	Role
`reference.wav`	Permitted reference recording
`reference.txt`	Exact transcript of the reference
`targets.csv`	Fixed target sentences for every compared run
`run_metadata.json`	Model, device, software, language, settings
`outputs/*.wav`	Generated speech, scored and inspected

Controlled comparison: keep target text and scoring rule fixed; change one factor at a time (clip, model, language, instruction).

The TTS Problem Contract

The TTS Contract

Text-to-speech system
A model maps target text y, control info c, and optional reference speech x^{ref} to a waveform:

$$\hat{x} \sim p_\theta(x \mid y, c, x^{\text{ref}}).$$

Voice cloning
TTS conditioned on reference speech from a target speaker, aiming to preserve timbre, accent, pitch range, and speaking style.

"Cloning" is not a guarantee — a short clip may capture timbre but lose emotion or pronunciation habits; long targets may drift.

Four Evaluation Dimensions From One Equation

Symbol	Running-example meaning	Evaluation question
y	Target sentence in `targets.csv`	Were the intended words spoken?
c	Language and style controls	Was the requested style followed?
x^{ref}	Reference voice clip	Does output resemble the speaker?
$\hat{x}$	Generated waveform	Clean, natural, fast to produce?
θ	Pretrained model parameters	Which model version produced this?

Output is often non-deterministic: record seed and decoding settings; note offline vs streaming mode (different latency budgets).

Audio as Data, Reused for TTS

Reusing Audio Vocabulary From ASR

Same objects: waveform samples, sample rate, frames, log-mel features.

Direction reverses: ASR consumes audio; TTS *produces* it.

$$\text{duration} = \frac{\text{samples}}{\text{sample rate}} = \frac{480{,}000}{24{,}000} = 20 \text{ s}.$$

Duration is budget — every extra second adds samples, context, decoding, and latency.

Reference Audio Checks Before Cloning

Problem	Why it matters	Practical check
Long silence	Wasted context, poor timing	Inspect duration before/after trim
Clipping	Distorts timbre	Look for repeated peaks at max amplitude
Wrong transcript	Weak text-audio alignment	Read while listening once
Background noise	Copied into outputs	Record in a quiet room
Multiple speakers	Confuses identity	Use single-speaker clip

Make audio preprocessing explicit. Record resampling, trimming, and loudness normalization as part of the run, not invisible cleanup.

Text, Pronunciation, and Prosody

Text Is More Than Characters

Text normalization
Convert written text to a spoken form: numbers, abbreviations, dates, units, symbols.

Example: "Dr. Lee read 3.14 kg on 04/05/2026."

- Is "Dr." *Doctor* or *Drive*?
- "3.14 kg" = "three point one four kilograms"?
- Date locale: April 5 or May 4?

Grapheme vs phoneme
Graphemes are written units; phonemes are sounds. "read" has two pronunciations distinguished only by tense.

Prosody and Style

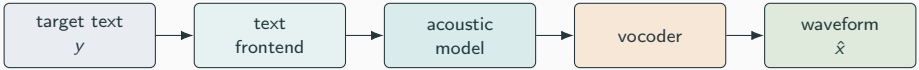
Prosody
Pattern of timing, rhythm, pitch movement, stress, and pausing. Two utterances can share words yet differ substantially.

- "This model is fast" — statement, surprise, or skeptical question.
- Punctuation is a weak prosody cue, not a complete spec.
- Modern TTS may accept natural-language style instructions (e.g. "speak slowly, like explaining to a new learner").

Make the text easy before making it clever. Add equations, names, and units *one at a time* so you can attribute failures.

The Classical Neural TTS Pipeline

Stages of a Classical Pipeline



Vocoder
A model or algorithm that maps an acoustic representation (mel spectrogram or discrete codes) into waveform audio.

Tacotron 2: seq2seq mel + WaveNet vocoder. FastSpeech: parallel mel + duration predictor + length regulator. VITS & HiFi-GAN: end-to-end and adversarial.

Length Regulator and Duration Loss

Phoneme i has hidden h_i and predicted duration d_i:

$$(h_1, \ldots, h_L) \longrightarrow (\underbrace{h_1, \ldots, h_1}_{d_1}, \ldots, \underbrace{h_L, \ldots, h_L}_{d_L}).$$

Total acoustic frames: $\sum_i d_i$. Duration loss against teacher alignments:

$$\mathcal{L}_{\text{dur}} = \sum_{i=1}^{L} (\log \hat{d}_i - \log d_i)^2.$$

Explicit duration improves speed, robustness, and rate control; replaces drifting attention with parallel decoding.

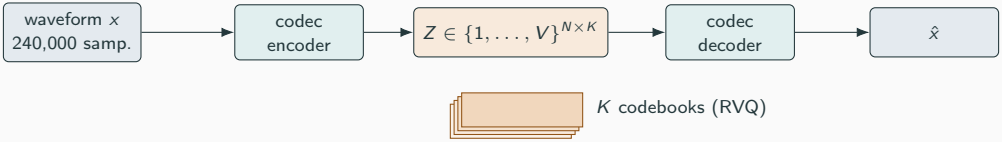
Failure Modes by Stage

Stage	Question answered	Common failure if weak
Text frontend	How to speak the text?	Wrong number, abbrev., or name
Alignment/duration	How long should each unit last?	Skipped or repeated words
Acoustic model	Which features match the text?	Robotic tone, weak prosody
Vocoder	Features $\rightarrow$ waveform	Noise, metallic, muffled

Useful debugging starts by naming the stage where the evidence points — not by replacing the largest model.

Speech Tokens, Codecs, and Codec LMs

Why Tokens? Closing the Sequence-Length Gap



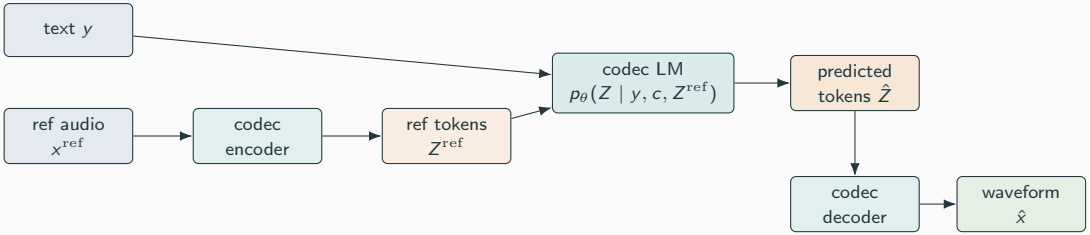
10 s at 24 kHz = 240,000 samples; a sentence is tens of text tokens. Residual VQ: first codebook coarse, later codebooks refine timbre. Examples: SoundStream, EnCodec.

Token Scale and Latency

Representation	Scale for 10 s	Tradeoff
Waveform 24 kHz	240,000 samples	Precise but very long
Mel spectrogram	Hundreds of frames	Compact but continuous
12.5 Hz, 16-codebook	125 frames, 2,000 entries	Short discrete; needs decoder

Tokens compress, not solve. Decoder must still recover timing, timbre, and intelligibility. Streaming decoders that need future context add first-packet latency.

Codec Language Model



$$\mathcal{L} = -\sum_{n=1}^{N}\sum_{k=1}^{K} \log p_\theta(Z_{n,k} \mid y, c, Z^{\mathrm{ref}}, Z_{<n,\cdot}, Z_{n,<k}).$$

Factorized over time n and codebooks k.

Transformer TTS

Two Generation Patterns

Autoregressive $p_\theta(z_n \mid z_{<n}, y, c, Z^{\mathrm{ref}})$. Flexible, familiar, but each step waits for previous step.

Parallel / non-autoregressive predicts many units at once; faster but must still preserve alignment and timing.

Acoustic prompt
Reference audio (or its tokens) used to condition speech generation, supplying speaker and style information not in the text.

VALL-E: codec LM with short acoustic prompts for unseen-speaker TTS.

Streaming and First-Packet Latency

First-packet latency
Time from receiving enough input to begin synthesis until the first playable audio packet. A streaming metric, *not* total runtime.

- Live tutoring, dialogue, voice assistant: first-packet latency dominates.
- Overnight audiobook: total throughput and quality dominate.
- Larger model $\Rightarrow$ better quality but more memory and slower.
- Lower token rate $\Rightarrow$ shorter sequences but less acoustic detail.
- More sampling $\Rightarrow$ variety, but instability risk.

Identify a setting that is measured to work for a specific task.

Qwen3-TTS Case Study

Qwen3-TTS: Modes

Multilingual, controllable, streaming TTS family with 3-second voice cloning, description-based control, Apache 2.0 license.

- **25 Hz tokenizer:** single codebook, diffusion-transformer reconstruction.
- **12.5 Hz tokenizer:** multi-codebook, lightweight causal decoder for low-latency streaming.

Mode	Conditioning	Classroom use
Voice clone	text + ref audio + transcript	Voice-clone exercise
Custom voice	text + selected speaker	Baseline without ref clip
Voice design	text + voice description	Style exploration

Minimal Voice-Cloning Call

```
from qwen_tts import Qwen3TTSModel
model = Qwen3TTSModel.from_pretrained(
  "Qwen/Qwen3-TTS-12Hz-0.6B-Base", device_map="cuda:0")
wavs, sr = model.generate_voice_clone(
  text="A waveform is a sequence of audio samples.",
  language="English",
  ref_audio="reference.wav", ref_text="...")
```

The model call is one line of a larger experiment. A real run also records dtype, attention impl., device, package versions, manifest path, output dir, and wall-clock timing.

Voice Control and Cloning

Speaker Control vs Style Control

- Speaker control: output sounds like a selected/cloned voice.
- Style control: calm, excited, slow, formal — often interacts with speaker.
- Reference clip and reference transcript must *agree*; noisy conditioning $\rightarrow$ noisy output.

Speaker similarity
The degree to which generated speech matches the reference speaker, judged by listeners or measured by speaker embeddings.

Small-study rubric: 1 unlike, 2 weak, 3 clear, 4 strong, 5 nearly the same. Same listener, same rubric, all outputs.

Controlled Comparisons

Control choice	What changes	What stays fixed
Reference length	Speaker evidence	targets, model, language
Style instruction	Tone or rate	ref clip, target, model
Model size	Capacity and runtime	ref clip, target, instruction
Preset vs cloned	Speaker source	target text, rubric

Inspect every batch output. Average WER hides a single skipped “not” that reverses meaning. If text is rewritten, say so.

Evaluation, Latency, Errors

Content, Speed, Quality

TTS WER backcheck
Run ASR on generated speech, normalize hypothesis and target the same way, compute WER/CER. A content probe, not a complete TTS quality score.

Example: target "the model predicts speech tokens," ASR "the model predict speech token" $\Rightarrow$ WER $= 2/5 = 0.40$.

$$\text{RTF} = \frac{t_{\text{run}}}{t_{\text{audio}}}, \qquad \text{RTF} = \frac{4.5}{15} = 0.30 \text{ (faster than real time).}$$

Live tutor: first-packet latency. Audiobook: total throughput.

Speaker Similarity and MOS

Cosine similarity of speaker embeddings:

$$s_{\text{speaker}} = \frac{e_{\text{ref}}^{\top} e_{\text{gen}}}{\|e_{\text{ref}}\|_2 \|e_{\text{gen}}\|_2}.$$

Mean opinion score:

$$\text{MOS} = \frac{1}{M} \sum_{m=1}^{M} r_m, \qquad r_m \in \{1, \ldots, 5\}.$$

Caveats: embeddings depend on the speaker model; MOS depends on listener instructions, sample count, and prompt selection.

Run-Level Result Record

Field	Meaning	Example value
`model_id, mode`	Model id and mode	Qwen3-TTS 0.6B, voice clone
`ref_audio_seconds`	Reference duration	8.08 s
`language`	Target language	English
`target_count`	Sentences in manifest	10
`total_audio_seconds`	Generated duration	39.76 s
`wall_time_seconds`	Measured runtime	49.43 s
`rtf`	Run-level RTF	1.243
`wer, cer`	ASR-backcheck content	not scored
`speaker_similarity`	Listener/embedding score	not scored
`notes`	Failures and observations	timing baseline

Common TTS Failures

- Skipped or repeated words; wrong language drift.
- Incorrect number, date, unit reading.
- Unstable speaker identity; overacted emotion.
- Clipped audio; background noise copied from prompt.
- Unnatural pauses; rushed or labored pacing.

Each failure suggests a different next action: number errors $\rightarrow$ text normalization; speaker drift $\rightarrow$ cleaner ref or shorter target; slow $\rightarrow$ smaller model or optimized runtime; clipping $\rightarrow$ loudness check.

Practical Recipe

A Conservative First Workflow

Step	First choice
Reference clip	3–10 s, one speaker, clean, exact transcript
Target manifest	8–12 short sentences, fixed across runs
First run	Qwen3-TTS base voice-clone, default settings
Comparison	Change ONE factor: model, ref clip, instruction
Speed	Wall clock / generated audio duration
Quality	ASR-backcheck + listener scores + error notes

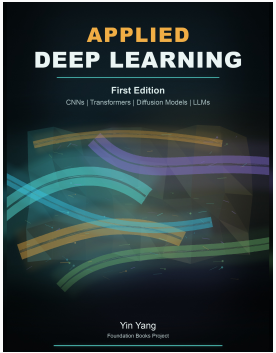
Troubleshoot locally: skipped words $\rightarrow$ length/punctuation; pronunciation $\rightarrow$ text norm; drift $\rightarrow$ ref clip; slow $\rightarrow$ RTF and backend; clipping $\rightarrow$ loudness.

Summary

Summary

- TTS is conditional generation: $\hat{x} \sim p_\theta(x \mid y, c, x^{\mathrm{ref}})$.
- Reuses ASR audio vocabulary; adds text norm, prosody, vocoders, acoustic prompts, and codec tokens.
- Classical pipeline: text frontend $\rightarrow$ acoustic model $\rightarrow$ vocoder.
- Modern transformer TTS predicts speech codec tokens, then decodes.
- Qwen3-TTS: voice clone, custom voice, voice design.
- Report: ASR-backcheck WER/CER, RTF, first-packet latency, similarity, naturalness, named failures.
- "Sounds like me" is a hypothesis — the report needs listening evidence, transcripts, and reproducible settings.

Next: efficient sequence architectures — recurrence, state-space models, efficient attention, MoE, and serving.



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	`foundation-books.github.io/applied-deep-learning`
Code	`github.com/foundation-books/applied-deep-learning-code`
Print	`amazon.com/dp/B0GZF7QRSH`
Kindle	`amazon.com/dp/B0GZF9221X`
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.